

Ostbayerische Technische Hochschule Amberg-Weiden
Faculty of Electrical Engineering, Media and Computer Science



Project Report
on
**Quantum Equation Retrieval & Relation
Mapping**

by
Kashif Riyaz

Degree program:	Masters in Artificial Intelligence for Industrial Applications
Exam ID:	Exam-NLP-S26-ID_12
Semester:	5 th
Supervisor:	Prof. Dr. Patrick Levi
Date:	June 25, 2026

Contents

1	Introduction	2
2	Key Ideas	2
3	Methodology	3
3.1	HTML Parsing	3
3.2	Description Extraction	4
3.3	Relation Classification	4
3.4	Audit Trail	5
4	Results and Discussion	5
4.1	Generalization	5
4.2	Quality	5
5	Conclusion	6
	References	7

1 Introduction

Quantum physics relies heavily on mathematics, and its papers are often packed with equations, of which typically only the significant ones are enumerated. Each equation has its own symbols, each carrying a particular meaning. Making sense of every symbol and equation by hand is feasible for one paper but the effort grows quickly across many. This project automates the extraction of enumerated equations, their symbols, and their meanings.

The system takes the ID of a quantum physics paper, downloads its content from arXiv, and for each enumerated equation extracts the equation, a short description, the meaning of its symbols, and how it relates to the other equations in the paper. Everything is stored in JSON, and an audit trail records which method produced each result so the extraction quality can be checked later.

For this prototype I used the papers in `paper_list_12`, in the given order, keeping the first seven numbered equations per paper. This required analysing 77 papers to reach 350 equations, with the last paper processed completely. The pipeline was initially designed around the $\text{\LaTeX}$ source, but since arXiv's `robots.txt` disallows it, all information is instead taken from the arXiv HTML [1], which provides the equations in a clean, structured form that is far more reliable to parse than the PDF.

2 Key Ideas

The task requires four items per equation: the equation, its description, its symbols, and its relations to other equations. The equation and symbols are structured objects, recovered directly from the HTML (where each carries its $\text{\LaTeX}$ form in a MathML attribute), whereas the description and relations must be inferred from natural language. The approach therefore splits the problem into a structural part and a language part.

For the **equations and symbols**, the idea is to rely on the structure arXiv's HTML already provides instead of recovering mathematics from a PDF. This was chosen because the HTML exposes the original $\text{\LaTeX}$ and clear equation markers, avoiding the unreliable and error-prone reconstruction of mathematics from a rendered PDF. Each equation, symbol, and reference is then replaced by a placeholder, since turning mixed math-and-text into clean tokens makes the later language processing far more robust.

For the **description**, the surrounding text (one sentence before and after, plus mention text for equations) is analysed grammatically rather than by keywords, because authors define notation in recognisable grammatical patterns, so the grammar carries the meaning more reliably than word proximity alone.

For the **relations**, a concept was derived because the task prescribes the grades but not how to assign them. The chosen principle, that two equations are related when they share something *specific to the paper*, was selected because it captures genuine connections while ignoring symbols that every equation trivially shares, which keeps the method general across papers.

No language model is prompted to generate text. Descriptions and relations come from structural rules and linguistic analysis; the only language model used is spaCy's `en_core_web_trf`, which serves purely to parse grammatical structure, never to produce content.

3 Methodology

The pipeline, shown in Figure 1, runs three major phases per paper.

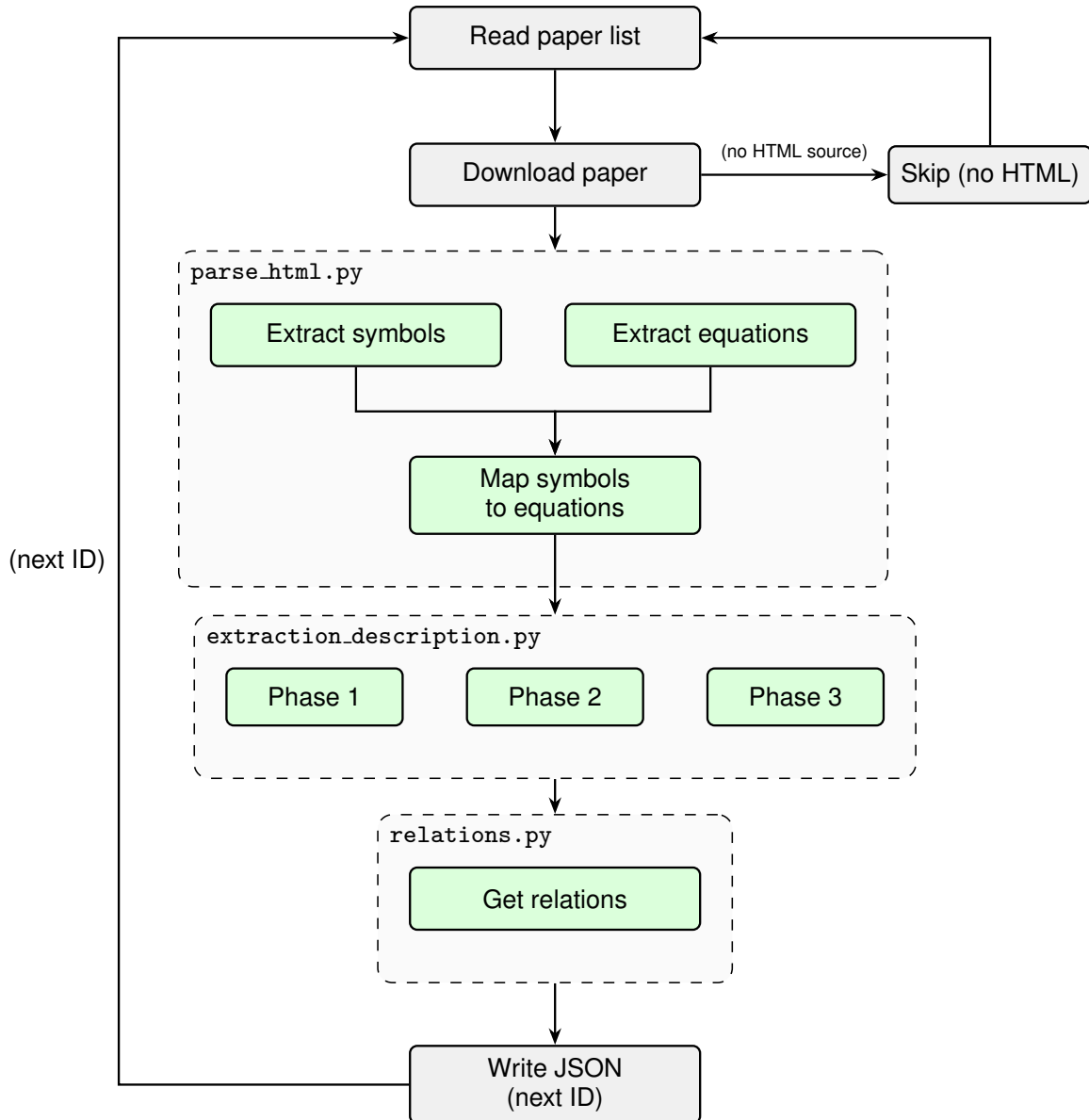


Figure 1: Flow chart of the whole pipeline.

3.1 HTML Parsing

An early attempt pulled all mathematics indiscriminately from the raw $\text{\LaTeX}$, but this captured too much noise. Parsed with BeautifulSoup [2], arXiv’s HTML wraps equations and symbols in distinct semantic tags, and leveraging these markers isolates only the meaningful components.

- **Symbol extraction:** the parser reads the $\text{\LaTeX}$ from the `altext` attribute of `<math>` tags and assigns each distinct symbol a token in order of appearance (`SYM1`, `SYM2`, ...); pure numbers are skipped.
- **Equation extraction:** enumerated equations are identified by their container attribute (e.g. `id="S1.E1"`) and labelled with their printed number, so equation (1) becomes `EQN1`. Non-enumerated or over-limit math becomes `TEMPEQN`, and a reference to an equation becomes `MEQN1`.
- **Symbol matching:** each symbol is linked to the equations it appears in, using boundary-aware matching so that a letter like `a` is not matched inside `\alpha`.

The module returns the clean placeholder text plus separate equation and symbol dictionaries, from which the symbol-to-equation mapping is built.

3.2 Description Extraction

Each symbol and equation needs a short description. Several approaches were tried and rejected: keyword proximity (too vague), cosine and attention scoring over word vectors (vector closeness does not identify the defining phrase), and NER. The final approach analyses the *grammatical structure* of the surrounding sentences (split into sentences with NLTK [3]) using the spaCy transformer [4] `en_core_web_trf` purely as a parser, in three phases.

Phase 1 – grammar rules (high confidence). The symbol's dependency relation is inspected to find the defining noun phrase, via patterns such as a copula (" ψ is the wave function"), a passive clause ("the amplitude is given by A "), an appositive (" A , the amplitude"), or a prepositional governor ("the wave function for ψ ").

Phase 2 – proximity fallback (low confidence). When no pattern applies, nearby noun phrases are scored by direction and distance; since authors usually name a concept before its symbol, noun phrases to the *left* are weighted higher.

Phase 3 – LHS substitution (equations only). For a low-confidence equation, it is split at "=" and its left-hand side compared to the paper's symbols; a matching symbol's description is reused. Equations sharing the same left-hand side then receive one unified description, so a low-confidence equation inherits a high-confidence one's description.

All candidates are cleaned and filtered to reject vague heads ("function", "equation"), references, citations, numbers and pronouns.

3.3 Relation Classification

The task defines a graded relation ("strong", "potential", "none") between every pair of equations. The concept adopted is that two equations are related when they share something *specific to the paper* rather than something every equation trivially has in common. Each pair is graded on shared symbols, with shared description words as a fallback.

Filtering. Only informative symbols count as evidence: *ubiquitous* symbols (in $\geq 80\%$ of equations), *non-variable* tokens (numbers, operators, constants such as `\pi`), and bare *single-letter* variables (weak evidence only) are excluded.

Grading. A pair is **strong** if it shares a distinctive multi-character variable, or if one equation defines a symbol used by the other; **potential** if it shares only a single common letter, or if their descriptions share a meaningful word; and **none** otherwise.

3.4 Audit Trail

Every step writes a short note to a per-equation audit trail, so each saved value can be traced back to the exact text and rule that produced it. This makes the dataset checkable and demonstrates compliance with the rules, since every value is shown to come from arXiv text rather than from generation. Each step adds its own entries:

- **Equation extraction** saves the captured $\text{\LaTeX}$ read straight from the HTML.
- **Symbol matching** logs the symbols linked to each equation, e.g. for $P_{\{r\}}=P_{\{0\}}$ IS the matched symbols $P_{\{r\}}$, $P_{\{0\}}$ and r .
- **Description extraction** logs the context sentence, chosen description, firing rule and confidence, e.g. “photon creation operator (rule=copula, conf=high)”. In the logged context only the symbol of interest is shown in $\text{\LaTeX}$ (in $\$ \dots \$$); others stay as codes (e.g. SYM9), keeping clear which token the description belongs to.
- **Relation classification** logs each graded pair, e.g. “ $1 \rightarrow 3$: strong (shares $I(x, t)$)”.

Because it records the rule and confidence, the trail directly supports the error analysis in Section 4.

4 Results and Discussion

The dataset contains 350 equations. Of the 78 papers processed, 16 had no arXiv HTML source and 7 contained no enumerated equations, leaving 55 papers that contributed equations.

4.1 Generalization

The approach should transfer well to arbitrary quantum physics papers because it relies on universal signals, not paper-specific tuning. Equation and symbol extraction depend only on arXiv’s HTML markup, identical across all rendered papers, so the structural results below should hold generally; the only hard limit is papers with no HTML. The description rules use grammatical patterns common to physics writing, and the relation concept uses generic properties, neither hardcoded to these papers.

4.2 Quality

Equation extraction: Extraction was fully reliable, every stored item was a genuinely enumerated equation, captured correctly with zero false positives. This is because arXiv’s HTML wraps each enumerated equation in an explicit tag that is unambiguously parsed, separating it from inline or unnumbered mathematics without error-prone text matching.

Symbol extraction and matching. Standalone extraction was accurate, and matching succeeded in almost all cases: only 4 of 350 equations (1.1%) ended with no matched symbols. Failures occur when a symbol carries slightly different $\text{\LaTeX}$ inside the equation than as

a standalone token, typically extra sub/superscripts, so a literal comparison fails (e.g. a stored $\dots^{\wedge}\{A^{\wedge}\{n\}\text{to } X_{\{n\}}\}$ versus $\dots^{\wedge}\{A^{\wedge}\{n\}\backslash\text{to } X_{\{n\}}\}$, differing by one backslash). This is hard to fix cleanly: normalising the $\text{\LaTeX}$ would recover some cases but risks merging genuinely different symbols, so the conservative exact match was kept.

Description quality. Semantic fields are the weakest part, with prompting disallowed, a meaning is recoverable only if stated in reachable text. Across the dataset, 57% of symbol descriptions came from high-confidence grammar rules and 43% from the low-confidence fallback. Symbol descriptions beat equation descriptions, of which 6.3% have no usable description, almost all proof or derivation steps. It works when the meaning is stated plainly (equation (1) of arXiv:2405.07909: $\hat{a}^{\dagger}(\omega_s) \rightarrow$ “photon creation operator”, conf=high), and fails otherwise (equation (3) of arXiv:2510.12951: the fallback returned “Appendix B”, latching onto a reference instead of the quantity).

These limits are intrinsic to the rule-based design: adding rules becomes case-wise and breaks cases that already worked. Real improvement would need a prompting-based model with a fixed prompt, which the rules forbid.

5 Conclusion

This project built a prototype that extracts enumerated equations, their symbols, descriptions, and relations from quantum physics papers on arXiv, and stores them in a graph-ready JSON dataset with an audit trail. The structural parts, equation and symbol extraction, worked very reliably because they use arXiv’s HTML markup, while the descriptions are limited by the no-prompting rule and by how clearly each author defines their notation. Where the task left choices open, we used the HTML source instead of the disallowed $\text{\LaTeX}$ source and adopted a shared-symbol concept for grading relations. Because the approach relies on general structural and grammatical signals rather than paper-specific tuning, it should also work on other quantum physics papers.

AI tool use declaration. Claude Opus was used to help refine wording and to add docstrings to the code. It was not used to generate any part of the dataset; all equation, symbol, description, and relation extraction was performed by the rule-based pipeline without any prompting of language models.

References

- [1] Cornell University, *arXiv.org e-print archive*, <https://arxiv.org>, Accessed: 2026.
- [2] L. Richardson, *Beautiful Soup documentation*, <https://www.crummy.com/software/BeautifulSoup/>, 2007.
- [3] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009.
- [4] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, “spaCy: Industrial-strength natural language processing in python,” Zenodo, 2020. DOI: 10.5281/zenodo.1212303.